

1. Code to Test Error Handling and Data Extraction

- **Modified app.py:** Here's the updated Flask code, incorporating the error handling and regex improvements we discussed. I've added comments to help you understand the changes.

```
from flask import Flask, request, render_template, jsonify
```

```
import os
```

```
import tempfile
```

```
from pdf2image import convert_from_path
```

```
import pytesseract
```

```
import re
```

```
app = Flask(__name__, template_folder='templates')
```

```
# (Optional) Tesseract path - Modify if needed
```

```
# pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.exe'
```

```
def extract_data_from_pdf(pdf_path):
```

```
    """
```

```
    Extracts data from a PDF file.
```

```
    Args:
```

pdf_path (str): Path to the PDF file.

Returns:

dict: A dictionary containing the extracted 'name_of_work' and 'estimate_amount'.

Returns None for a field if extraction fails.

Raises:

Exception: If any error occurs during PDF processing.

"""

try:

*pages = convert_from_path(pdf_path, 300) # Reduced DPI for
potential speed improvement; adjust as needed*

text = ""

for page in pages:

text += pytesseract.image_to_string(page) # Corrected function

name

name_of_work = extract_name_of_work(text)

estimate_amount = extract_estimate_amount(text)

*return {"name_of_work": name_of_work, "estimate_amount":
estimate_amount}*

except Exception as e:

```
        raise Exception(f"Error processing PDF: {e}") # Improved error
message
```

```
def extract_name_of_work(text):
```

```
    """
```

Extracts the "Name of Work" from the given text.

Args:

text (str): The text extracted from the PDF.

Returns:

str: The extracted "Name of Work", or "Not Found" if not found.

```
    """
```

```
    match = re.search(r'(?:(?:Name of Work|Work
Description):?\s*(.*?)(?=\n|\Z|Estimate)', text, re.DOTALL |
re.IGNORECASE)
```

```
    if match:
```

```
        return match.group(1).strip()
```

```
    return "Not Found"
```

```
def extract_estimate_amount(text):
```

```
    """
```

Extracts the "Estimate Amount" from the given text.

Args:

`text (str)`: The text extracted from the PDF.

Returns:

`str`: The extracted "Estimate Amount", or "Not Found" if not found.

"""

```
match = re.search(r'(?:(?:Estimate Amount|Estimated  
Cost):?\s*([₹Rs.]\s*[\d.,]+(?: Lakhs?| Crores?))?)', text, re.IGNORECASE)
```

if match:

```
    amount = match.group(1).strip()
```

```
    if re.search(r'[\d.,]', amount):
```

```
        return amount
```

```
    return "Not Found"
```

```
return "Not Found"
```

```
@app.route('/', methods=['GET'])
```

```
def upload_form():
```

"""

Renders the file upload form.

"""

```
return render_template('upload.html')
```

```
@app.route('/extract', methods=['POST'])
```

```
def extract():
```

```
    """
```

```
    Handles PDF file upload and data extraction.
```

```
    """
```

```
    if 'pdf_file' not in request.files:
```

```
        return render_template('result.html', error='No file uploaded',  
status=400)
```

```
    pdf_file = request.files['pdf_file']
```

```
    if pdf_file.filename == ":
```

```
        return render_template('result.html', error='No filename provided',  
status=400)
```

```
    if pdf_file.filename.rsplit('.', 1)[1].lower() != 'pdf:
```

```
        return render_template('result.html', error='Invalid file type. Please  
upload a PDF file', status=400)
```

```
    try:
```

```
        with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as  
temp_pdf:
```

```

temp_pdf.write(pdf_file.read())

pdf_path = temp_pdf.name

extracted_data = extract_data_from_pdf(pdf_path)

return render_template('result.html', data=extracted_data,
status=200)

except Exception as e:

    # Log the error (important for debugging in production)

    print(f'Error: {e}')

    return render_template('result.html', error=str(e), status=500) #
Internal Server Error

finally:

    if 'pdf_path' in locals():

        os.remove(pdf_path)

if __name__ == '__main__':

    app.run(debug=True)

```

- *templates/upload.html (No Changes Needed):*

- This file remains the same as it provides the HTML form for uploading the PDF.

```
<!DOCTYPE html>

<html>

<head>

    <title>PDF Data Extractor</title>

</head>

<body>

    <h1>Upload a PDF File</h1>

    <form method="POST" action="/extract" enctype="multipart/form-
data">

        <input type="file" name="pdf_file" required><br><br>

        <input type="submit" value="Extract Data">

    </form>

</body>

</html>
```

- **templates/result.html (Modified to display "Not Found"):**

```
<!DOCTYPE html>

<html>

<head>

    <title>Extraction Result</title>
```

```

</head>

<body>

  <h1>Extraction Result</h1>

  {% if error %}

    <p style="color: red;">Error: {{ error }}</p>

  {% elif data %}

    <p>Name of Work: {% if data.name_of_work != "Not
Found" %}{{ data.name_of_work }}{% else %}<b>Not Found</b>{%
endif %}</p>

    <p>Estimate Amount: {% if data.estimate_amount != "Not
Found" %}{{ data.estimate_amount }}{% else %}<b>Not Found</b>{%
endif %}</p>

  {% else %}

    <p>No data extracted or an error occurred.</p>

  {% endif %}

  <br>

  <a href="/">Upload Another PDF</a>

</body>

</html>

```

2. Creating Different PDF Formats for Testing

- It's tricky to provide actual PDF files directly within this format. However, I can guide you on how to create variations for testing.

You can use a Word processor (like Microsoft Word, if available on your office laptop, or LibreOffice Writer) to create documents and then save them as PDFs. Alternatively, online tools can convert documents to PDF.

- Here's a breakdown of PDF variations and how to create them:

* **2.1 Basic Structure**

- * Create a simple Word document with the following structure:

Name of Work: Construction of a Bridge

Estimate Amount: Rs. 1,23,45,678

- * Save it as a PDF. This will be your base case.

* **2.2 Variations in "Name of Work"**

- * **Variation 1: Extra Spaces:**

Name of Work : Construction of a Bridge

- * **Variation 2: Newline Character:**

Name of Work:

Construction of a Bridge

* **Variation 3: Different Keyword**

Work Description: Construction of a Bridge

* **Variation 4: Multiline Name**

Name of Work: Construction of a
long and complex bridge over the river

* **2.3 Variations in "Estimate Amount"**

* **Variation 1: No "Rs."**

Estimate Amount: 1,23,45,678

* **Variation 2: Using "₹"**

Estimate Amount: ₹ 1,23,45,678

** **Variation 3: Using "Lakhs" or "Crores"***

Estimate Amount: 123.45 Lakhs

Estimate Amount: 12.34 Crores

** **Variation 4: No Commas***

Estimate Amount: Rs 12345678

** **2.4 Edge Cases***

** **Missing Data:** Create a PDF where either "Name of Work" or "Estimate Amount" is missing.*

** **Incorrect Formatting:** Intentionally misformat the data (e.g., "Estimate Amount: ABC")*

** **Extra Text:** Add extra text before or after the fields you're extracting.*

** **Different Fonts:** Use different fonts in your Word documents before converting to PDF.*

** **Images:** Include images in your PDF to see how OCR handles them. This can sometimes introduce noise.*

** **Tables:** If your PDFs contain tables, create a sample with a table containing the data. This is a **very** important edge case, as OCR can struggle with tables.*

** **Scanned PDFs:** If possible, get a scanned PDF (an image of a document). OCR is designed for this, but accuracy can vary. You can create a scanned PDF by printing a document and then scanning it back in.*

- **Saving as PDF:**
 - **Microsoft Word:** In Word, go to "File" -> "Save As" and choose "PDF" as the file format.
 - **LibreOffice Writer:** In Writer, go to "File" -> "Export As" -> "Export as PDF".
 - **Online Tools:** Search for "online document to PDF converter." There are many free tools where you can upload your document and download the PDF. *Be cautious about uploading sensitive documents to online tools.*

Key Points

- **Thorough Testing:** The more varied your test PDFs, the more confident you can be in your application's robustness.
- **Iterative Improvement:** Test your code with each PDF variation, analyze the results, and refine your regular expressions or code as needed.
- **Logging:** The `print(f'Error: {e}')` line in the `extract` function is crucial. In a real application, you'd use a proper logging library (like

Python's logging module) to write errors to a file, making debugging much easier.